

Deployment Report: Smart UI Auditor

Product: AI-Powered Smart UI Auditor (desktop + local API)

Repository root: `uI` workspace

Document version: 1.0 | **Date:** April 2026

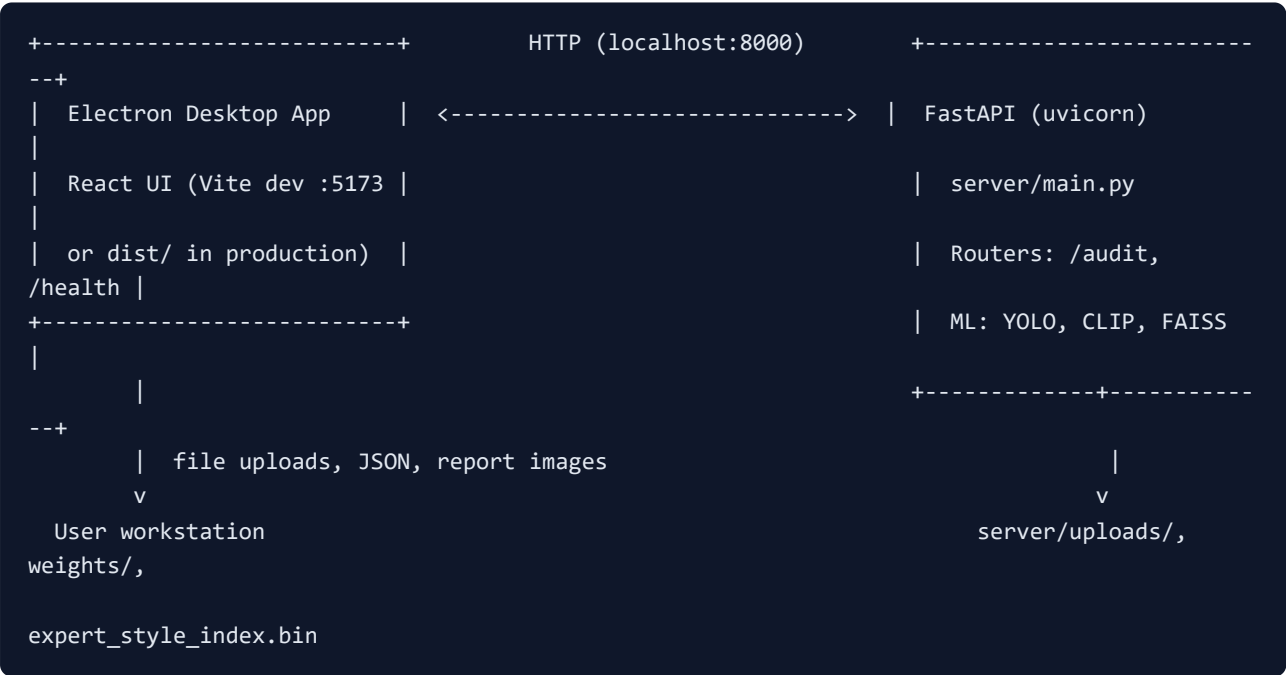
Classification: Internal / technical operations

STANDALONE APP This solution is deployed as a **standalone desktop application** using **Electron**. The UI runs inside a native window (not a public multi-tenant SaaS). Core auditing does **not** require a cloud UI host: the client talks to a **local FastAPI** service on the same machine (default `http://localhost:8000`). ML models, FAISS index, and uploads stay on the operator’s workstation or your controlled server. Optional features (e.g. Google Gemini) only need network access for those specific API calls.

1. Executive summary

The system has two runtime parts: (A) a **Python FastAPI** backend with PyTorch, Ultralytics YOLO, OpenAI CLIP, FAISS, and related services under `server/`; (B) an **Electron + React + Vite** desktop shell under `ai_powered_ui_auditor/`. End users run the desktop app and keep the API process running locally (development today uses two terminals; production can wrap both in an installer or process supervisor). This report covers prerequisites, artefacts, ports, environment variables, runbooks, and hardening for deployment.

2. Architecture overview



3. Technology inventory (verified from repo)

Layer	Technology	Notes
API	FastAPI, Uvicorn	ASGI app in <code>server/main.py</code>
ML / CV	PyTorch, torchvision, Ultralytics YOLOv8	Detector weights: <code>server/weights/best.pt</code>
Embeddings	OpenAI CLIP (ViT-B/32)	<code>server/services/vectorizer.py</code>
Vector search	faiss-cpu	<code>server/expert_style_index.bin</code> + <code>server/image_paths.txt</code>
Image I/O	OpenCV, Pillow	BGR crops, annotated PNG/JPEG reports
Desktop	Electron 28, React 18, Vite 5, TypeScript	<code>ai_powered_ui_auditor/package.json</code>
Optional	google-genai, SMARTUI_RL, DeepFace, etc.	Feature-specific; see <code>requirements.txt</code>

4. Standalone application model

4.1 What “standalone” means here

- **Desktop shell:** Electron provides a self-contained windowed app (`app/main.js`). In production mode it loads the built UI from `dist/index.html` from disk—no public hosting of the SPA is required.
- **Local API:** The React/Electron front end is configured to call `http://localhost:8000` (see e.g. `ElementAuditPage.tsx` , hooks). All classic audit traffic stays on loopback unless you intentionally change the base URL.
- **On-prem data:** Screenshots and reports are written under `server/uploads/` ; no third party receives them for the CLIP+FAISS path.

4.2 Current packaging gap (honest)

Today’s repo scripts start **Vite + Electron** in dev and expect the Python server separately. A fully single-file “double-click installer” would bundle Python/uvicorn or compile services—document as a roadmap item. Until then, deployment is **two local processes**, which is still standalone from a **no shared cloud UI** perspective.

5. Prerequisites

Requirement	Version / detail
Python	CPython 3.12 or 3.13 (standard build) — avoid Python 3.13 “free-threading”/nogil for PyTorch (see startup message in <code>server/main.py</code>).
Node.js + npm	LTS recommended (for Electron and Vite).
GPU	Optional; CUDA speeds CLIP/YOLO if drivers and torch build match.
Disk	Space for PyTorch wheels, CLIP weights, YOLO weights, FAISS index, and uploads.

6. Backend deployment

6.1 Install Python dependencies

From the repository root (parent of `server`):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r ai_powered_ui_auditor\requirements.txt
```

Adjust path if your requirements file is consolidated elsewhere. Ensure CLIP installs from the Git URL listed in requirements.

6.2 Required artefacts

- `server/weights/best.pt` — YOLO weights (must exist for detection).
- `server/expert_style_index.bin` — FAISS index for similarity audit.
- `server/image_paths.txt` — paths aligned with index rows.
- Expert image library on disk matching those paths (e.g. `Expert_Library/...`).

If the FAISS index is missing, the server still runs but similarity scores degrade to zero with “N/A” matches—document this for operators.

6.3 Start the API

```
cd <repo-root>
.venv\Scripts\activate
uvicorn server.main:app --host 127.0.0.1 --port 8000
```

Use `--host 0.0.0.0` only on trusted LANs with firewall rules. Restrict CORS in `server/main.py` before any non-localhost exposure (currently `allow_origins=["*"]` for development).

6.4 Health check

GET `/health` returns `{"status":"ok"}` — suitable for load balancers and monitoring.

7. Desktop client deployment

7.1 Development mode (two processes)

```
Terminal 1: uvicorn server.main:app --reload --port 8000
Terminal 2: cd ai_powered_ui_auditor
npm install
npm run dev
```

`npm run dev` runs Vite on port 5173 and launches Electron loading `http://localhost:5173` (`app/main.js`).

7.2 Production-style desktop build

```
cd ai_powered_ui_auditor
npm install
npm run build
```

Then run Electron with `NODE_ENV=production` so the window loads `dist/index.html` from disk. Package with electron-builder or your org's signing pipeline (not included in this repo by default).

8. Configuration and secrets

- `GEMINI_API_KEY` — read from `ai_powered_ui_auditor/.env` via `server/config.py` for Google GenAI features. **Do not commit real keys.** Use OS secret store or deployment vault in production.
- Front end currently hard-codes `http://localhost:8000`; for packaged installers, introduce `VITE_API_BASE_URL` at build time.

9. Networking and ports

Port	Service	Bind
8000	Uvicorn / FastAPI	127.0.0.1 (recommended default)
5173	Vite dev server	localhost (dev only)

10. Static panel and desktop downloads

If `panel-research-site/` exists, it is mounted at `/panel`. Endpoint `GET /downloads/desktop` serves the newest installer from `panel-research-site/releases/` (.exe, .msi, .zip, .dmg).

11. Security checklist (pre-production)

- Replace wildcard CORS with explicit origins.
- Rate-limit upload size and MIME types on `/audit` routes.

- Run API behind HTTPS reverse proxy if remote access is required; use auth tokens.
- Rotate API keys; never log secrets.
- Sanitize filenames in uploads and report URLs.
- Document data retention for screenshots in `server/uploads/`.

12. Monitoring and operations

- Probe `/health` every 30–60s.
- Disk alerts on `server/uploads` growth.
- Log uvicorn access and application errors to rotated files.
- Version-tag Docker images or release bundles with Git commit SHA.

13. Rollback

Keep previous `best.pt`, FAISS index, and Python wheel lockfile. Roll back by restoring artefacts and restarting uvicorn. For desktop, ship previous signed installer in `releases/`.

14. Troubleshooting

Symptom	Likely cause	Mitigation
PyTorch import error	Unsupported Python build	Use standard CPython; reinstall torch from official index.
Audit returns zero similarity	Missing FAISS index	Deploy <code>expert_style_index.bin</code> + <code>image_paths.txt</code> .
Electron blank screen in prod	dist not built	Run <code>npm run build</code> ; set <code>NODE_ENV=production</code> .
CORS blocked	API not on localhost	Align front-end base URL or tighten CORS origins explicitly.

15. Sign-off

Prepared by: _____ **Role:** _____

Reviewed by: _____ **Date:** _____

PDF export: Open this HTML file in Microsoft Edge or Google Chrome. Press Ctrl+P, choose *Save as PDF*, paper size A4, default margins. This produces the deployment report PDF without extra tools.